CAPSTONE SYSTEM DOCUMENTATION

SMMG Healthcare Appointment Management System

Architecture, roles, data model, core workflows, deployment, integrations, and verification guide for the current implementation.

Technology: Nuxt 4 · Vue 3 · FastAPI · PostgreSQL · MinIO · Resend · External AI API
Deployment: Docker Compose
Document date: August 2026

1. System Overview

SMMG provides online healthcare appointment coordination and an AI-powered health and first-aid assistant. It supports four account roles while keeping authorization in the FastAPI backend.

Patient services

Doctor discovery, availability, booking, rescheduling, cancellation, notifications, profile management, and AI conversations.

Clinical workflow

Doctor appointment lists, schedule visibility, status updates, professional profiles, and personal reports.

Operations

Staff dashboards and appointment coordination across pending, confirmed, completed, and cancelled records.

Administration

User, doctor, staff, catalog, schedule, report, and AI conversation management.

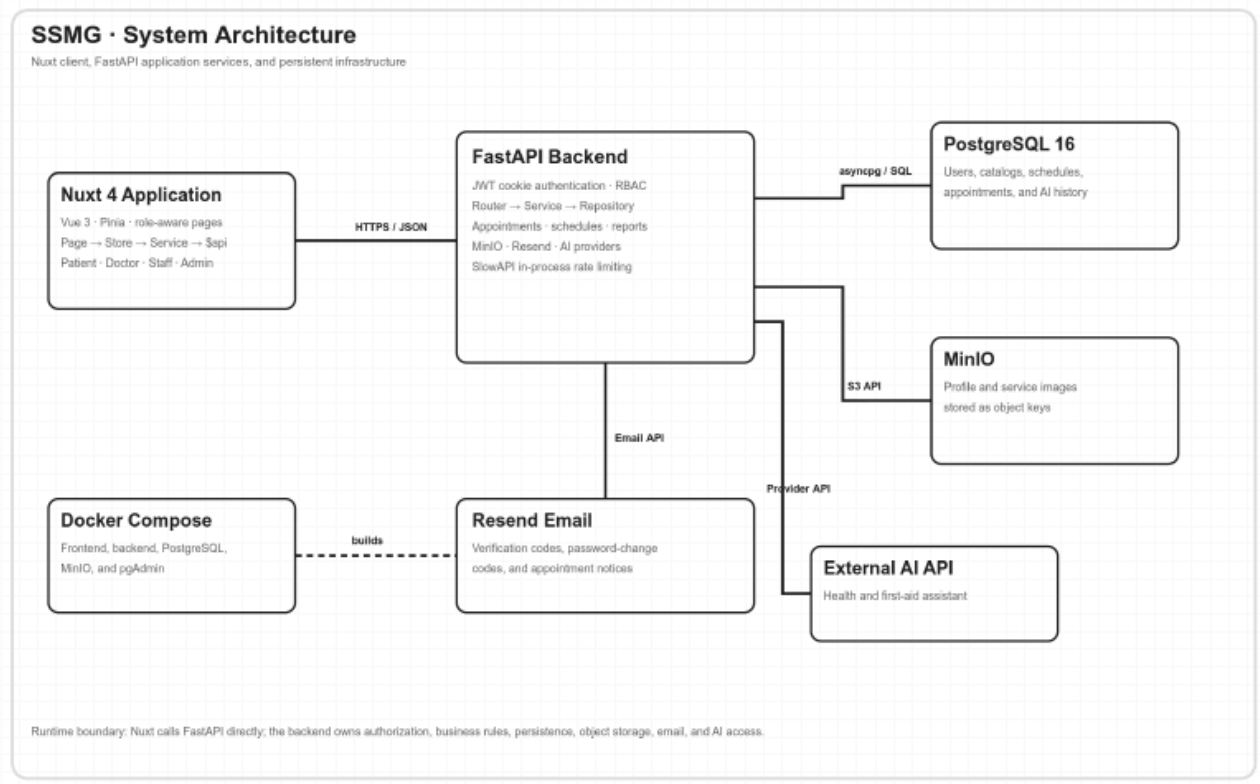
Design boundaries

- Nuxt calls FastAPI directly; there is no backend-for-frontend proxy.
- JWT cookie authentication and backend role checks protect private operations.
- PostgreSQL is the source of truth for users, catalogs, schedules, appointments, and AI history.
- MinIO credentials remain in the backend; browser uploads pass through FastAPI.
- SlowAPI rate limiting runs inside FastAPI without Redis.

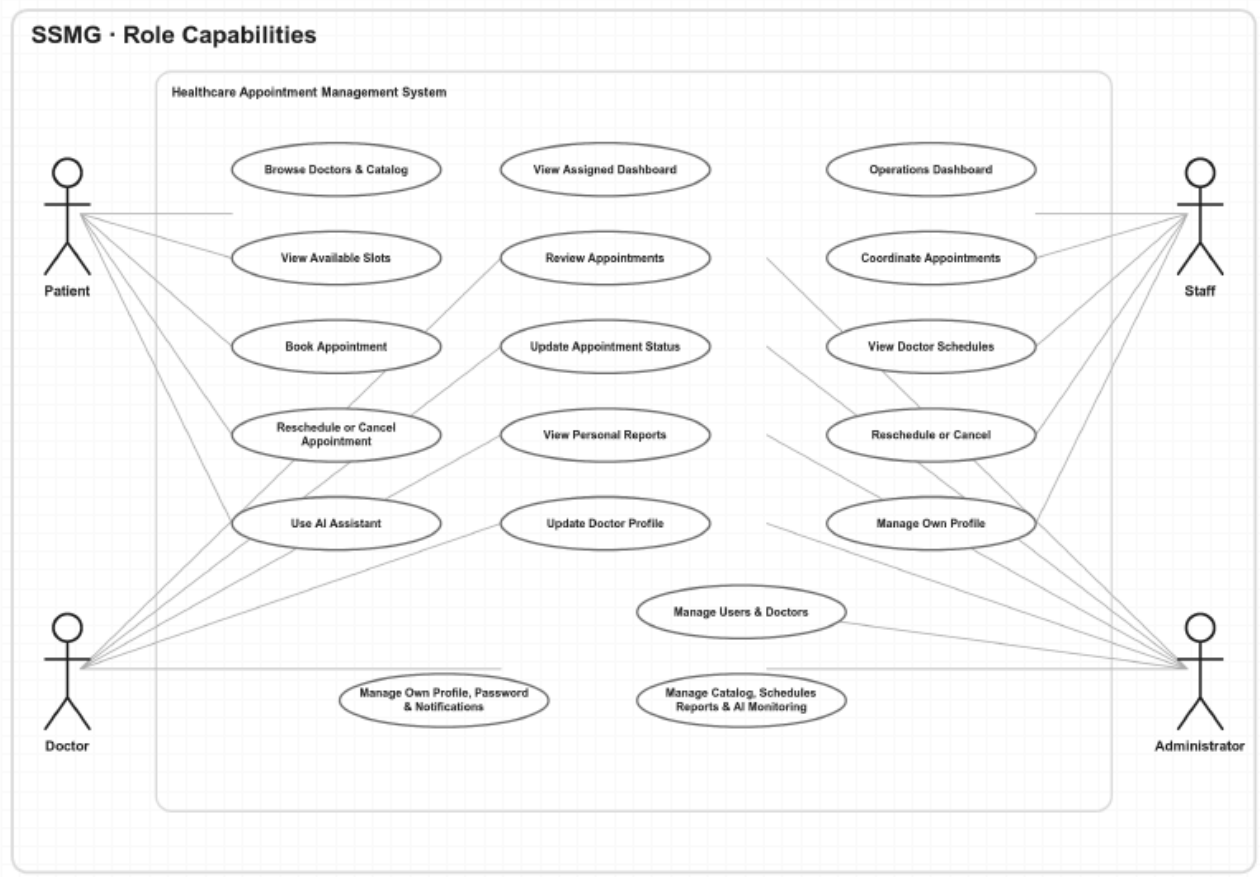
Project layout

Path	Responsibility
`smmgfrontend`	Nuxt pages, layouts, Pinia stores, API services, and shared TypeScript types.
`smmgbackend`	FastAPI routers, services, repositories, SQLAlchemy models, providers, and seed scripts.
`docker-compose.yml`	Frontend, backend, PostgreSQL, MinIO, and pgAdmin services.

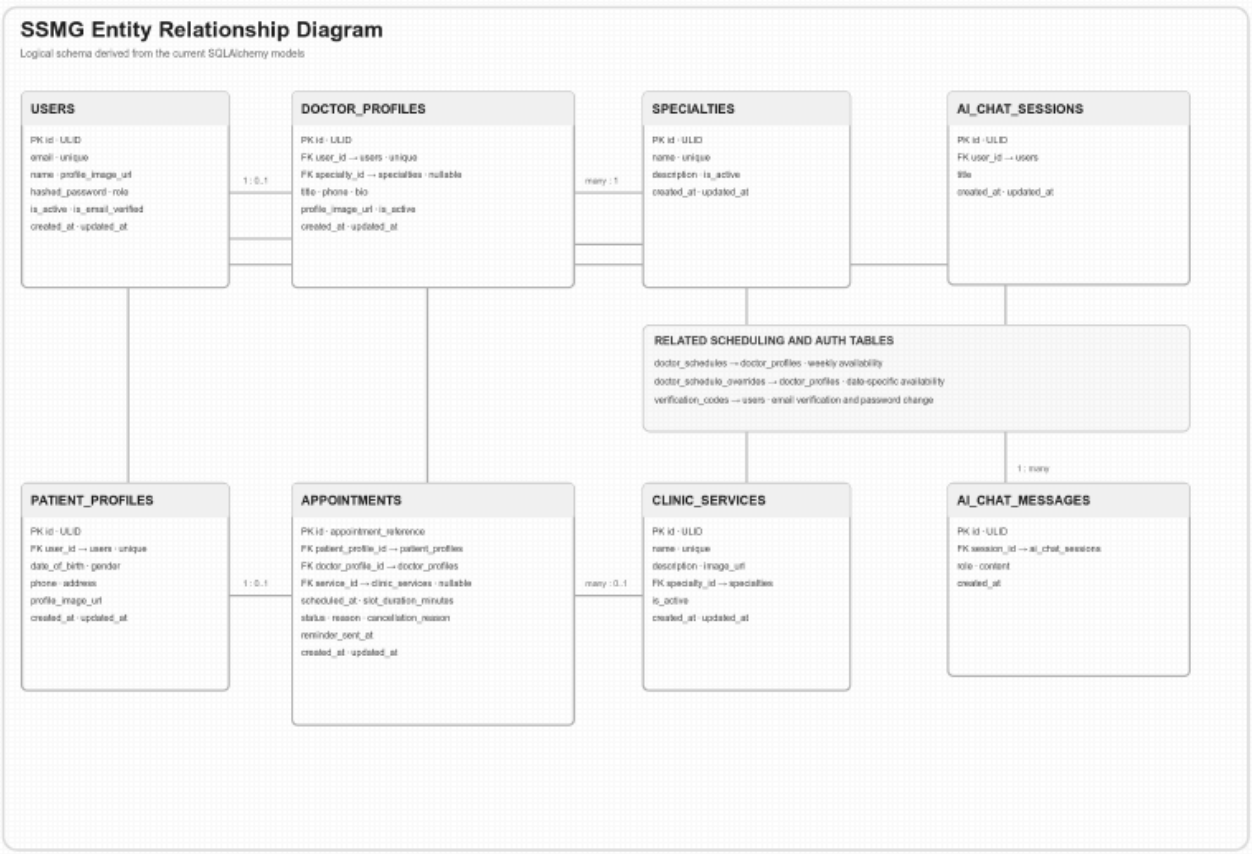
2. System Architecture



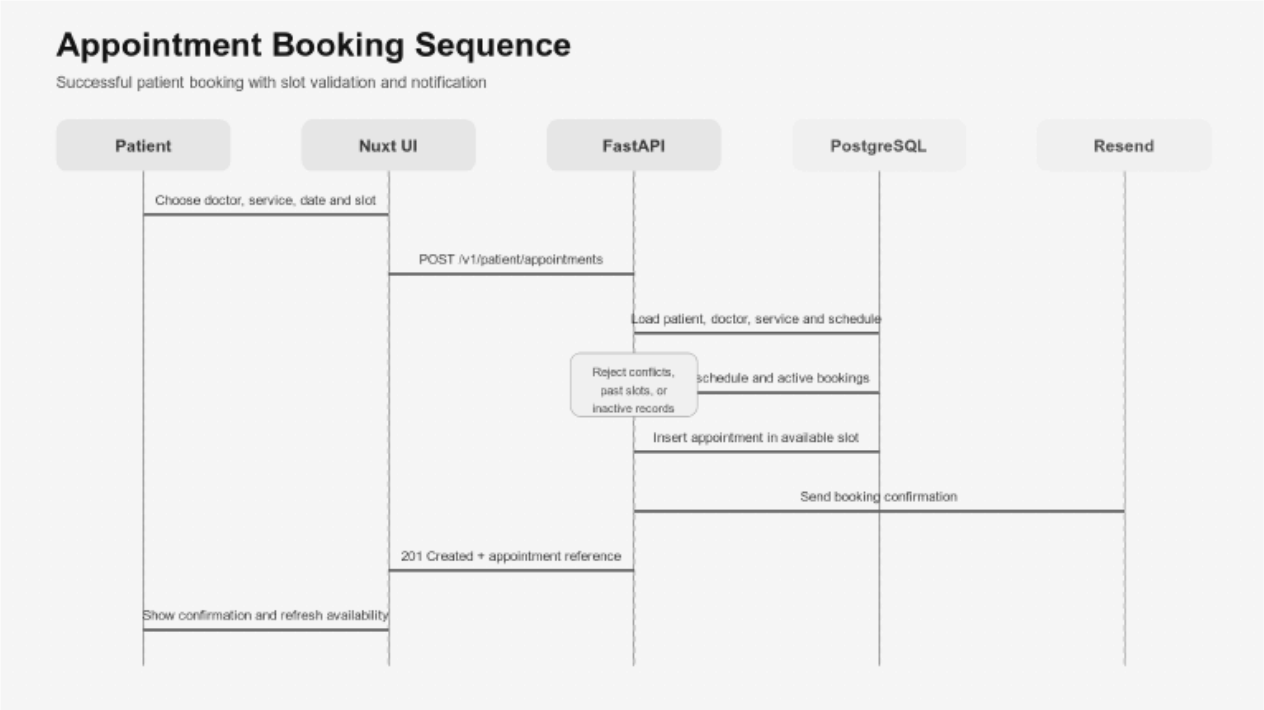
3. Role Capabilities



4. Data Model

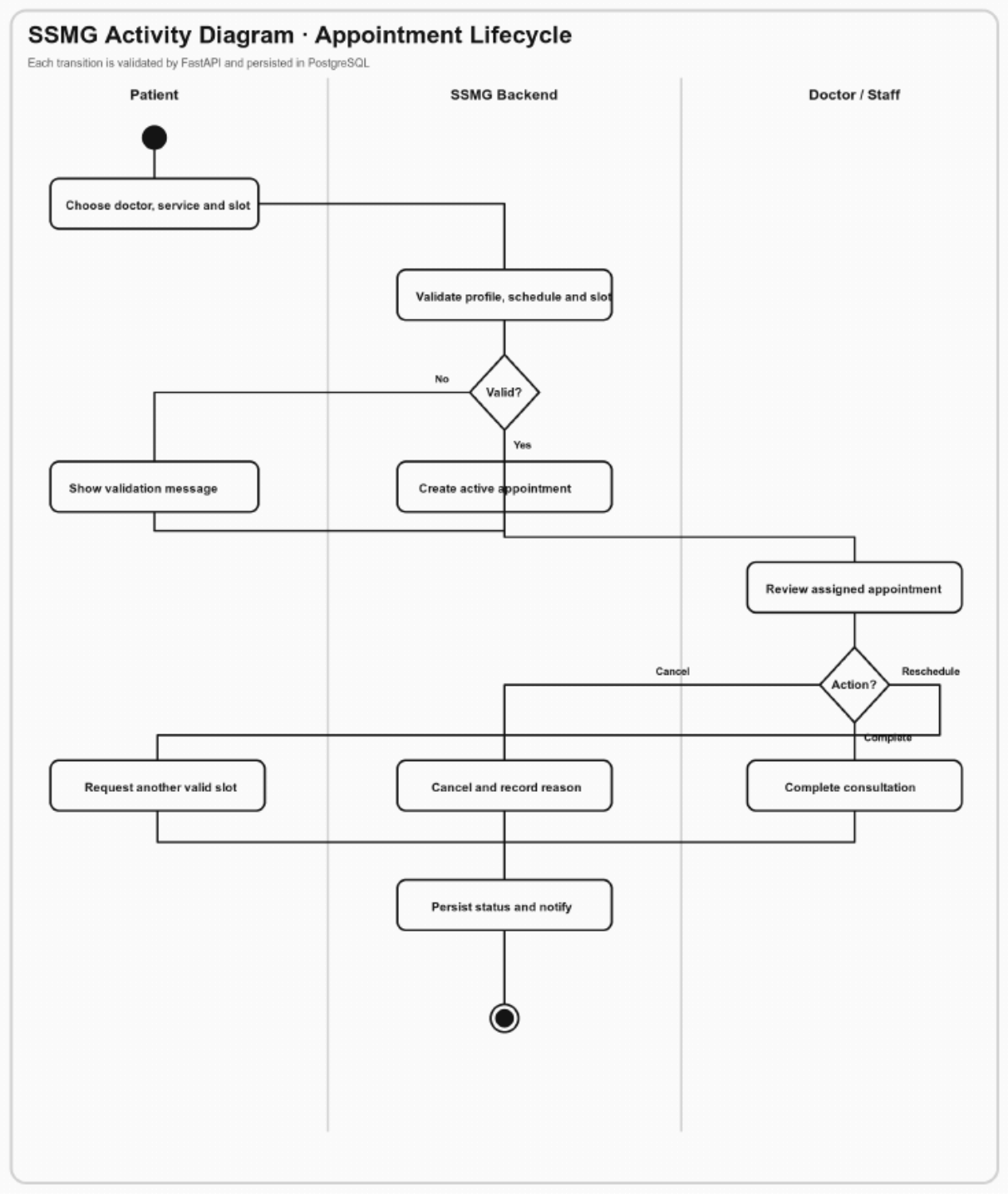

SSMG Entity Relationship Diagram
Logical schema derived from the current SQLAlchemy models
USERS
PK id · ULID
email · unique
name · profile_image_url
hashed_password · role
is_active · is_email_verified
created_at · updated_at
1 : 0..1
DOCTOR_PROFILES
PK id · ULID
FK user_id → users · unique
FK specialty_id → specialties · nullable
title · phone · bio
profile_image_url · is_active
created_at · updated_at
many : 1
SPECIALTIES
PK id · ULID
name · unique
description · is_active
created_at · updated_at
AI_CHAT_SESSIONS
PK id · ULID
FK user_id → users
title
created_at · updated_at
RELATED SCHEDULING AND AUTH TABLES
doctor_schedules → doctor_profiles · weekly availability
doctor_schedule_overrides → doctor_profiles · date-specific availability
verification_codes → users · email verification and password change
1 : many
PATIENT_PROFILES
PK id · ULID
FK user_id → users · unique
date_of_birth · gender
phone · address
profile_image_url
created_at · updated_at
1 : 0..1
APPOINTMENTS
PK id · appointment_reference
FK patient_profile_id → patient_profiles
FK doctor_profile_id → doctor_profiles
FK service_id → clinic_services · nullable
scheduled_at · slot_duration_minutes
status · reason · cancellation_reason
reminder_sent_at
created_at · updated_at
many : 0..1
CLINIC_SERVICES
PK id · ULID
name · unique
description · image_url
FK specialty_id → specialties
is_active
created_at · updated_at
AI_CHAT_MESSAGES
PK id · ULID
FK session_id → ai_chat_sessions
role · content
created_at

5. Appointment Booking



> The database has a partial unique index for active doctor/time combinations. This protects a slot from concurrent pending or confirmed bookings.

6. Appointment Lifecycle



7. Backend Modules and APIs

Module	Primary responsibility
Authentication	Registration, email verification, login/logout, current account, profile update, and verified password change.
Users	Role-filtered account listing, staff creation, and account activation controls.
Doctors and patients	Role profiles, public doctor catalog, and profile updates.
Specialties and services	Database-backed healthcare catalog management.
Schedules	Weekly doctor availability, date overrides, and available slot calculation.
Appointments	Booking, listing, status transitions, cancellation, and rescheduling.
Dashboard and reports	Role-specific summaries and aggregate operational reports.
AI integration	User chat sessions/messages and administrator monitoring.
Uploads	Validated image upload and object retrieval through MinIO.
Notifications	In-app appointment notices and email provider coordination.

Backend layering

HTTP handling remains in routers, workflow and authorization logic belongs to services, and SQLAlchemy access belongs to repositories. MinIO, Resend, and AI access use provider modules.

API discovery

When the backend is running, interactive OpenAPI documentation is available at `http://localhost:8000/docs`.

8. Local Deployment and Verification

Services

Service	Address
Nuxt frontend	`http://localhost:3000`
FastAPI	`http://localhost:8000`
OpenAPI	`http://localhost:8000/docs`
MinIO console	`http://localhost:9001`
pgAdmin	`http://localhost:5050`

Startup

Run `docker compose up -d --build` from the repository root. Backend startup executes the idempotent admin, catalog, and workforce seed scripts before Uvicorn starts.

Verification checklist

- Confirm all containers are running with `docker compose ps`.
- Open the FastAPI documentation and confirm the API schema loads.
- Sign in with each role and verify role-specific navigation.
- Book an available appointment and verify it appears for the patient and assigned doctor.
- Upload a profile or service image and confirm the object is present in MinIO.
- Register PostgreSQL in pgAdmin using host `postgres`, database `smmg`, and port `5432`.

> The documented credentials are local demonstration defaults. They must not be used for a public deployment.